

YouClone

YouTube Clone Project Documentation

Complete Interview Preparation Guide

Django 6.0 | Python 3.13 | Bootstrap 5

Prepared for Technical Interviews

Live Demo: <https://streamhub.pythonanywhere.com/>

Table of Contents

1	Project Overview
2	Project Objectives
3	Key Features
4	System Architecture
5	Technology Stack
6	Database Design
7	Application Workflow
8	Challenges & Solutions
9	Deployment Process
10	Interview Questions & Answers
11	Key Talking Points

1. Project Overview

YouClone is a full-stack video sharing platform inspired by YouTube, built as a portfolio project to demonstrate proficiency in modern web development with Python and Django. The platform enables users to upload, view, share, and interact with video content through a feature-rich interface that mirrors core YouTube functionality.

The application supports complete user authentication with custom profiles, video management (upload, edit, delete with drag-and-drop), channel-based content organization, social interactions (likes, dislikes, comments, subscriptions), search with query history, tag-based content recommendations, and personal watch history tracking. All views use Django's class-based views with server-rendered HTML templates styled with Bootstrap 5 in a custom dark theme.

The project is deployed live on PythonAnywhere (streamhub.pythonanywhere.com) and is also configured for deployment on Vercel with PostgreSQL. With approximately 10,000 lines of code across 120+ files organized into 9 Django apps, this project demonstrates a well-structured, production-grade Django application following MVT (Model-View-Template) architecture.

2. Project Objectives

Primary Goals

- **Build Core YouTube Functionality:** Implement video upload, playback, search, and social interactions that mirror the YouTube experience.
- **Demonstrate Django Mastery:** Showcase expertise in Django 6.0 including custom user models, class-based views, signals, authentication, and ORM.
- **Production-Ready Deployment:** Configure and deploy the application to a live server with proper security settings, static/media file handling, and database configuration.
- **Responsive & Accessible UI:** Create a polished dark-mode interface using Bootstrap 5 that works across devices.

Learning Objectives

- Implement a custom user model extending Django's AbstractUser
- Build a multi-app Django project with clean separation of concerns
- Handle file uploads (video, images) securely
- Implement many-to-many relationships and unique constraints
- Create dynamic, interactive templates with vanilla JavaScript
- Configure production deployment with SSL, CSRF, and security best practices

3. Key Features

- **User Authentication & Profiles:** Custom user model with profile pictures, bio, and registration date. Signup with avatar upload and password strength indicator. Login with password visibility toggle. Automatic channel creation via Django signals on user registration.
- **Video Management:** Full CRUD for videos with drag-and-drop upload interface, thumbnail preview, character counters for title/description, progress bar animation, and tag selection. Videos are stored via FileField and served directly.
- **Video Playback & Interactions:** HTML5 video player with controls and poster image. Like/dislike toggle system (click to toggle on/off; one active reaction per user per video). View counter incremented on each detail page visit.
- **Channel System:** Every user gets an auto-created channel. Channel detail pages with banners, avatars, subscriber/video/view counts, tab switching between Videos and About sections, and owner management links.
- **Subscription System:** Toggle subscribe/unsubscribe with one click. Self-subscription prevention via model validation. Dedicated subscription feed page showing subscribed channels.
- **Comment System:** Full CRUD for comments on videos. Inline comment form on video detail page. Edit with original comment preview. User's personal comment management page.
- **Search with History:** Search videos by title and channels by name using Django ORM icontains queries. Automatic search history tracking with individual delete and clear-all functionality.
- **Tag-Based Recommendations:** Video tagging system with many-to-many relationship via VideoTagMap. Tag pages aggregating all videos for a tag. User interest scoring for potential future recommendations.
- **Watch History:** Automatic recording of video views with timestamps. Date-grouped history display (Today, Yesterday, Older). Individual watch history entry removal and full clear.
- **Related Videos:** Sidebar on video detail page showing up to 10 videos from the same channel.
- **Liked Videos Collection:** Dedicated page displaying all videos the user has liked, with heart animation hero section.
- **Responsive Dark Theme:** Full dark mode with custom CSS variables (--bg-color, --surface-color, --primary-color). Collapsible sidebar, adaptive grid layouts, smooth CSS animations, and mobile-responsive design.

4. System Architecture

Architecture Pattern: Django MVT (Model-View-Template)

The application follows Django's standard MVT pattern with 9 loosely-coupled apps, each responsible for a specific domain. The architecture emphasizes separation of concerns, code reusability, and maintainable organization.

High-Level Architecture

```

Browser --> Django URL Router --> App Views (Class-Based) --> Models/ORM --> Database
|
+--> Templates (Django Template Language)
+--> Static Files (CSS, JS, Images)
+--> Media Files (User Uploads)

```

App Breakdown

- **django_project/:** Project settings, root URL configuration, WSGI/ASGI entry points, logging filters.
- **accounts/:** Custom User model (CustomUser), signup view/form, authentication URLs.
- **channels/:** Channel model (OneToOne with User), CRUD views, auto-creation signal.
- **videos/:** Video, VideoReaction, VideoView models; video CRUD, likes, liked-videos list.
- **comments/:** Comment model, full CRUD views, inline creation on video pages.
- **subscriptions/:** Subscription model with unique constraint, toggle subscribe view.
- **search/:** Search view querying videos/channels; SearchHistory model with management.
- **recommendations/:** VideoTag, VideoTagMap, UserInterest models; tag-based video listing.
- **watch_history/:** WatchHistory model auto-tracked on video view; history management views.
- **pages/:** Home/landing page view (TemplateView).

Key Architectural Decisions

- **Class-Based Views throughout:** All views use Django's generic CBVs (ListView, DetailView, CreateView, UpdateView, DeleteView) for consistency, less boilerplate, and built-in form handling.
- **Signals for cross-app concerns:** Channel auto-creation on user signup uses Django's `post_save` signal, keeping concerns decoupled from the accounts app.
- **Server-rendered templates:** No SPA framework; all HTML is rendered server-side with Django Template Language, simplifying the architecture and avoiding client-side complexity.
- **Media separation:** User-uploaded content (videos, thumbnails, banners, avatars) stored in `/media/` subdirectories, separate from application code.

5. Technology Stack

- **Backend Framework:** Django 6.0.3 - Latest Django release with modern ORM, class-based views, built-in auth, and admin interface.
- **Python:** 3.13 - Latest Python version providing performance improvements and new language features.
- **Database (Dev):** SQLite - Lightweight file-based database for local development with zero configuration.
- **Database (Production):** PostgreSQL via DATABASE_URL on Vercel, or SQLite on PythonAnywhere.
- **CSS Framework:** Bootstrap 5.3.3 with custom dark theme CSS variables, responsive grid, and utility classes.
- **Icons:** Bootstrap Icons 1.11.3 - SVG-based icon library with 1,900+ open-source icons.
- **Typography:** Google Fonts - Outfit (headings) + DM Sans (body) for modern, clean typography.
- **Image Processing:** Pillow >= 10.0 - Python Imaging Library fork, required by Django's ImageField.
- **Form Rendering:** django-crispy-forms 2.x + crispy-bootstrap5 2024.x - Elegant Bootstrap 5 form rendering.
- **Static Files (Prod):** WhiteNoise - Serves static files efficiently in production without a separate web server.
- **Hosting:** PythonAnywhere (current live), Vercel (configured) - Two deployment targets with different database backends.
- **Version Control:** Git/GitHub - Full commit history with structured project development.

Why Django 6.0?

Django 6.0 is the latest major release, offering improved performance, enhanced security defaults, better async support, and updated ORM features. Using the latest version demonstrates awareness of current Django ecosystem and the ability to work with modern tooling.

6. Database Design

The database consists of 11 models across 9 apps, designed to normalize data and enforce referential integrity through Django's ORM relationships. Below is each model with its key fields.

accounts.CustomUser

Extends AbstractUser

- profile_picture: ImageField (nullable) - User avatar
- bio: TextField - User biography/description
- created_at: DateTimeField - Account creation timestamp
- email: EmailField - Marked as unique

channels.Channel

OneToOneField to User

- owner: OneToOneField(User) - Each user has exactly one channel
- name: CharField(255) - Channel display name
- description: TextField - Channel description
- banner: ImageField (nullable) - Channel banner image
- created_at: DateTimeField - Channel creation date

videos.Video

Core content model

- uploader: ForeignKey(User) - Video owner
- channel: ForeignKey(Channel) - Associated channel
- title: CharField(255) - Video title
- description: TextField - Video description
- video_file: FileField - Uploaded video file
- thumbnail: ImageField (nullable) - Video thumbnail
- views: PositiveIntegerField - View count (default=0)
- duration: IntegerField - Duration in seconds (default=0)
- uploaded_at: DateTimeField - Upload timestamp

videos.VideoReaction

User reactions to videos

- user: ForeignKey(User)
- video: ForeignKey(Video)
- reaction: CharField(10) - choices: like/dislike
- UniqueConstraint: (user, video) - One reaction per user per video

videos.VideoView

View tracking

- user: ForeignKey(User, nullable, SET_NULL)

- video: ForeignKey(Video)
- viewed_at: DateTimeField(auto_now_add)

comments.Comment

Video comments

- video: ForeignKey(Video)
- user: ForeignKey(User)
- text: TextField - Comment content
- created_at: DateTimeField
- updated_at: DateTimeField

subscriptions.Subscription

Channel subscriptions

- user: ForeignKey(User)
- channel: ForeignKey(User)
- subscribed_at: DateTimeField
- UniqueConstraint: (user, channel) - No duplicate subscriptions
- clean(): Validates user != channel (prevents self-subscription)

search.SearchHistory

User search queries

- user: ForeignKey(User)
- query: CharField(255)
- searched_at: DateTimeField(auto_now_add)

recommendations.VideoTag

Content tags

- name: CharField(100) - Unique tag name

recommendations.VideoTagMap

Video-tag mapping

- video: ForeignKey(Video)
- tag: ForeignKey(VideoTag)
- UniqueConstraint: (video, tag)

recommendations.UserInterest

User tag interests

- user: ForeignKey(User)
- tag: ForeignKey(VideoTag)
- score: IntegerField(default=0)
- UniqueConstraint: (user, tag)

watch_history.WatchHistory

Viewing history

- user: ForeignKey(User)

- video: ForeignKey(Video)
- watched_at: DateTimeField(auto_now_add)
- watch_duration: IntegerField(default=0) - Seconds watched

7. Application Workflow

User Journey: First Visit

1. User visits the homepage (/) - sees hero section with platform overview and CTA buttons.
2. User signs up at /accounts/signup/ - fills in username, email, password, optionally uploads profile picture and bio. A Channel is automatically created for the user via post_save signal.
3. User logs in at /accounts/login/ - redirected to homepage with personalized greeting.
4. User browses videos at /videos/ - sees paginated video grid with filter chips.
5. User clicks a video - navigates to /videos/<pk>/ with player, reactions, comments, and related videos.

Video Upload Flow

1. User navigates to /videos/create/
2. Selects video file via drag-and-drop or file picker (progress bar shown)
3. Fills in title (with character counter), description (with character counter)
4. Uploads thumbnail image with preview
5. Selects tags from available checkboxes (tag preview updates in real-time)
6. Submits form - video is saved and user redirected to video detail page
7. View count begins at 0, incremented on each visit to detail page

Social Interaction Flow

1. On video detail page, user can like/dislike (toggle on/off, swap between reactions)
2. User can subscribe to the video's channel via toggle button (prevents self-subscription)
3. User can write a comment inline - comment appears immediately after submission
4. User can edit their own comments (with original text preview) or delete them
5. User's liked videos are collected at /videos/liked/ for easy access

Search & Discovery Flow

1. User enters query in search bar (available in navbar on all pages)
2. Results page shows matching videos (title icontains) and channels (name icontains)
3. Query is automatically saved to SearchHistory for that user
4. Recent searches displayed in sidebar - click to re-search or delete individual entries
5. User can browse tags at /recommendations/tag/<pk>/ to discover content by category
6. Watch history is automatically recorded and viewable at /watch_history/

8. Challenges & Solutions

Challenge: Like/Dislike Toggle Logic

Problem:

Users should be able to like (toggle on/off), dislike (toggle on/off), and swap between reactions seamlessly. If a user likes and then clicks like again, it should remove. If they like and click dislike, it should swap.

Solution:

Implemented a toggle-based reaction system using `get_or_create`. If a reaction exists for the (user, video) pair with the same reaction type, it's deleted (toggle off). If it exists with a different type, it's updated (swap). Otherwise, a new reaction is created. This provides YouTube-standard behavior.

Challenge: Channel Auto-Creation on Signup

Problem:

Every user should have a channel automatically without requiring manual setup. The channel and user are in different apps, creating a cross-app dependency.

Solution:

Used Django's `post_save` signal in `channels/signals.py` connected to User's `post_save` event. When a new User is created, the signal handler creates a Channel with default name matching the username. This keeps the accounts and channels apps loosely coupled.

Challenge: Self-Subscription Prevention

Problem:

Users should not be able to subscribe to their own channel. This requires validation at both the model and view level.

Solution:

Added a `clean()` method to the Subscription model that raises `ValidationError` if `user == channel`. The `ToggleSubscriptionView` checks this condition before processing and returns an appropriate message. The unique constraint on (user, channel) prevents duplicate subscriptions.

Challenge: Search Across Multiple Models

Problem:

Search needs to query both videos (by title) and channels (by name) simultaneously and display results in a unified page.

Solution:

Used Django ORM's `__icontains` lookup on both `Video.title` and `Channel.name`, executed as two separate queries. Results are passed to the template in a single context, with separate sections for video and channel results.

Challenge: Watch History Deduplication

Problem:

Each video view should be recorded without creating duplicate entries when users revisit the same video.

Solution:

Used `get_or_create()` in the WatchHistory model combined with updating the `watched_at` timestamp on re-views. This ensures only one history entry per (user, video) pair while keeping the timestamp current.

Challenge: Responsive Video Grid

Problem:

The video listing needs to look good on mobile, tablet, and desktop with different column counts.

Solution:

Used Bootstrap 5's responsive grid classes: row-cols-1 (mobile), row-cols-md-2 (tablet), row-cols-lg-3 (desktop), row-cols-xl-4 (large desktop). Combined with CSS Grid fallbacks and consistent card sizing.

Challenge: Production SSL and Security

Problem:

Deploying securely requires HTTPS, secure cookies, HSTS headers, and proper secret management.

Solution:

Configured `SECURE_SSL_REDIRECT`, `CSRF_COOKIE_SECURE`, `SESSION_COOKIE_SECURE`, `HSTS` settings. Secret key is loaded from environment variable with a fallback for development. Django's security middleware is fully enabled.

Challenge: Form Validation with File Uploads

Problem:

Video file uploads need size validation, format validation, and progress indication.

Solution:

Implemented client-side drag-and-drop with file size display and progress bar animation. Django's `FileField` and `ImageField` handle server-side validation. Forms include proper `enctype="multipart/form-data"` and accept attribute restrictions.

9. Deployment Process

Current Production (PythonAnywhere)

The application is live at <https://streamhub.pythonanywhere.com/> deployed on PythonAnywhere's free tier with the following setup:

- Python 3.13 virtual environment with all dependencies installed via pip
- WSGI file configured to serve the Django application from `django_project/wsgi.py`
- Static files collected via `python manage.py collectstatic`, served from `/static/` URL
- Media files (uploads) served from `/media/` URL with proper file mappings
- SQLite database (default) for simplicity on the free tier
- `DEBUG=False`, `ALLOWED_HOSTS` includes `streamhub.pythonanywhere.com`

Vercel Deployment (Configured)

The project is also configured for Vercel deployment with these additional considerations:

- Uses WhiteNoise for static file serving in production
- PostgreSQL database via `DATABASE_URL` environment variable
- Build commands: `collectstatic --noinput`, `migrate --noinput`
- Environment variables required: `SECRET_KEY`, `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`, `DATABASE_URL`

Deployment Steps (High Level)

1. Create PythonAnywhere account and set up a new Web app with manual configuration
2. Clone the repository to PythonAnywhere via Git or upload directly
3. Create and activate a Python 3.13 virtual environment, install `requirements.txt`
4. Run `python manage.py collectstatic` to gather static files
5. Run `python manage.py migrate` to apply all database migrations
6. Configure WSGI file to point to `django_project.wsgi.application`
7. Set up static and media file URL mappings in the Web tab
8. Set `DEBUG=False` and configure `ALLOWED_HOSTS` in environment variables
9. Reload the web app and verify functionality

Security Configuration

```
DEBUG = False
ALLOWED_HOSTS = ['streamhub.pythonanywhere.com', 'localhost', '127.0.0.1']
SECURE_SSL_REDIRECT = True
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
```

10. Interview Questions & Answers

Q: Tell me about your YouTube Clone project.

A: I built a full-stack video sharing platform called YouClone using Django 6.0 and Python 3.13. It has 9 Django apps handling user authentication, video upload/playback, channels, comments, subscriptions, search, tag-based recommendations, watch history, and the landing page. The frontend uses Bootstrap 5 with a custom dark theme and all pages are server-rendered using Django templates. It's deployed live on PythonAnywhere and also configured for Vercel. The project has about 10,000 lines of code across 120+ files.

Q: Why did you choose Django for this project?

A: Django was the ideal choice because it provides everything needed for a content-sharing platform out of the box: a powerful ORM for complex queries (search, filtering), built-in authentication system, admin interface for content management, file upload handling via FileField/ImageField, form validation, and class-based views that reduce boilerplate. Django 6.0 is the latest release with improved performance and security features. The MVT architecture also makes the code well-organized and maintainable.

Q: How did you handle video storage and delivery?

A: Videos are uploaded through a Django FileField with a drag-and-drop frontend interface. They're stored in the `/media/videos/` directory on the server filesystem. For the current deployment on PythonAnywhere, videos are served directly from the media URL. In production, this would ideally be offloaded to a CDN or cloud storage like AWS S3 for better performance. The current approach works well for a portfolio/demo project with reasonable traffic expectations.

Q: Explain the like/dislike system implementation.

A: The VideoReaction model stores user-video-reaction triplets with a unique constraint on (user, video) to enforce at most one reaction per user per video. The view uses a toggle-based approach: if a reaction exists with the same type, it's deleted (toggle off); if it exists with a different type, it's updated (swap); otherwise, a new reaction is created. This gives YouTube-standard behavior where clicking the same button removes your reaction and clicking the other button swaps it.

Q: How did you implement the search functionality?

A: Search uses Django ORM's `__icontains` lookup for case-insensitive substring matching. When a user submits a query, it searches both Video.title and Channel.name in parallel queries. Results are displayed in separate sections on the search results page. The query is automatically saved to SearchHistory using `get_or_create` to avoid duplicates. Users can view their recent searches, click to re-search, or delete individual entries.

Q: How does the subscription system work?

A: The Subscription model has ForeignKey fields to User (the subscriber) and User (the channel owner), with a unique constraint on (user, channel) to prevent duplicates. A `clean()` method validates that users cannot subscribe to themselves. The ToggleSubscriptionView uses a POST handler that checks if a subscription exists and either creates or deletes it. Channel detail pages show subscriber counts and the subscription list page shows all channels the user follows.

Q: What was the most challenging part and how did you solve it?

A: The most challenging aspect was handling the like/dislike toggle with proper state management across the three states (no reaction, liked, disliked). The solution involved using `get_or_create` with conditional logic to handle toggle-off, swap, and new creation cases. Another challenge was the cross-app signal for channel auto-creation, which required decoupling the accounts and channels apps while ensuring every user gets a channel on signup. Using Django's `post_save` signal with proper receiver registration solved this cleanly.

Q: How would you scale this application?

A: Several improvements would be needed for scale: 1) Offload video storage to a CDN like AWS S3 or CloudFront. 2) Add caching with Redis or Memcached for frequently accessed data. 3) Implement database read replicas for query-heavy operations. 4) Add a task queue (Celery) for async processing like video transcoding and thumbnail generation. 5) Implement lazy loading and pagination for all list views. 6) Consider a REST API with DRF if building mobile clients. 7) Add database indexing on frequently queried fields like title, `created_at`, and foreign keys.

Q: Why use class-based views instead of function-based views?

A: Class-based views offer several advantages: code reuse through inheritance, built-in generic behavior (list, detail, create, update, delete), cleaner separation of GET and POST handling, and mixins for cross-cutting concerns like login-required checks. For this project, CBVs reduced boilerplate significantly - for example, a `CreateView` handles form display, validation, save, and redirect with just a few lines of configuration. However, I also appreciate FBVs for their simplicity and explicitness in cases with complex custom logic.

Q: How did you ensure code quality and organization?

A: I followed Django best practices: each app has a single responsibility (accounts for auth, videos for video management, etc.), models use proper field types and constraints, views are class-based with appropriate mixins, templates use inheritance (`base.html` extended by all pages), and URL namespacing is consistent. The project uses Django's built-in test framework with test files in each app. Security settings are configured for production with HTTPS, secure cookies, and HSTS.

Q: Explain the database schema and relationships.

A: The database has 11 models across 9 apps. The core entities are User (`CustomUser` extending `AbstractUser` with profile fields), Channel (`OneToOne` to User, so each user has exactly one channel), and Video (`ForeignKey` to both User and Channel). Supporting models include `VideoReaction` (unique user-video constraint), `Comment` (`FK` to Video and User), `Subscription` (unique user-channel constraint with self-subscription prevention), `SearchHistory`, `VideoTag/VideoTagMap` (many-to-many between videos and tags), `UserInterest`, and `WatchHistory`. This design normalizes data while supporting all required queries efficiently.

Q: What security considerations did you implement?

A: Several security measures: CSRF protection on all forms (Django's built-in CSRF middleware), SQL injection prevention via Django ORM parameterized queries, XSS protection via Django template auto-escaping, password hashing with Django's default PBKDF2 algorithm, login-required decorators on protected views, owner-only access checks via `UserPassesTestMixin`, HTTPS redirect in production, secure cookie flags (`CSRF_COOKIE_SECURE`, `SESSION_COOKIE_SECURE`), HSTS headers, and `DEBUG=False` in production to avoid information disclosure.

11. Key Talking Points for Interview

Strengths to Emphasize

- End-to-end ownership: designed, built, and deployed the entire application independently
- Modern stack: Django 6.0 (latest), Python 3.13, Bootstrap 5
- Clean architecture: 9 loosely-coupled apps with single responsibility
- Production deployment: live on PythonAnywhere with proper security configuration
- Comprehensive feature set: 12+ features mimicking real YouTube functionality
- Code quality: 10,000 LOC, class-based views, template inheritance, proper model design

Areas for Future Improvement (Be Honest)

- REST API: No API endpoints; would add Django REST Framework for mobile/third-party access
- Async tasks: Video transcoding, email notifications would benefit from Celery
- Caching: No Redis/Memcached for performance optimization
- Testing: Basic test files exist but coverage could be expanded
- Video streaming: Currently serves full files; chunked streaming would be better
- CI/CD: No automated deployment pipeline configured yet

Impressive Metrics

- ~10,000 lines of code across 120+ files
- 9 Django apps, 11 database models, 40+ URL endpoints
- 2 deployment targets (PythonAnywhere live + Vercel configured)
- 24+ HTML templates with embedded CSS/JS interactivity
- Full CRUD operations for 5 entities (videos, channels, comments, subscriptions, tags)